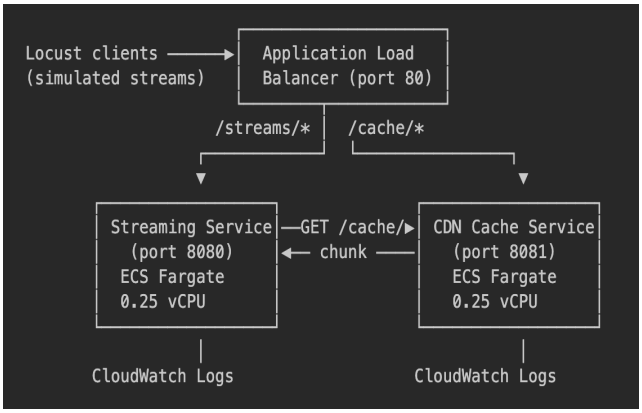
CS6650 - Distributed Music Streaming: Experiment Report

Team: Ananyaa Mylsamy · Manasha Sekar

System Overview

We built two Go microservices deployed on AWS ECS Fargate to simulate the core challenge of music streaming at scale: delivering fixed-size audio chunks to thousands of concurrent streams without rebuffering.



Architecture

Service	Role
Streaming Service	Manages active streams in-memory (sync.Map). On each chunk request, tries CDN first (500ms timeout); falls back to local CPU generation if CDN fails. Implements circuit breaker (5 failures -> open 30s) and adaptive quality switching.
CDN Cache Service	Pre-populates 100 songs × 300 chunks in-memory at startup. Serves chunks at low latency when the circuit is closed.

Audio is modelled as byte arrays: high-quality chunks (320kbps) cost ~500k CPU iterations; low-quality (128kbps) cost ~50k , a 10× difference that drives all three experiments.

Experiment 1 - Single-Instance Capacity Under Load

Purpose & Tradeoff: Find the maximum concurrent streams a single 0.25 vCPU Fargate instance can serve before p99 chunk delivery latency exceeds 500ms (the rebuffering threshold). The tradeoff is concurrency vs. latency: more users means more goroutines competing for CPU, which drives up per-chunk generation time.

Limitations: Single instance only; CDN circuit breaker trips at ~300 users in every run (forcing local fallback), so results reflect local CPU generation throughout - CDN hit performance is not measured here.

Results (AWS - Fargate, Single Instance, 0.25 vCPU)

Three runs at 300, 400, and 500 concurrent users (fixed 320kbps, adaptive=false, 3–5 min each):

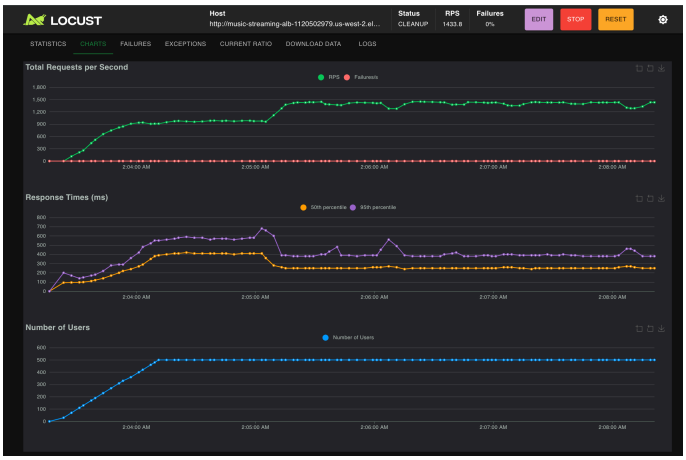
Users	Median (ms)	p95 (ms)	p99 (ms)	RPS	CDN Circuit
300	150	290	370	1,283	Open
400	220	410	480	1,447	Open
500	260	480	610	1,433	Open

p99 crosses 500ms at 500 users. RPS plateaus at 400->500 users, confirming CPU saturation.

500-user run - Statistics and Charts:



500 users' stats



500 user's charts

Analysis

The CDN circuit breaker opened in every run - at 300+ concurrent streams, CDN calls timed out consistently, forcing all chunks to be generated locally. This means the bottleneck is **CPU, not network**: each chunk costs ~500k CPU iterations, and with 500 goroutines competing on 0.25 vCPU, scheduling pressure drives p99 above threshold. The RPS plateau at 1,433 between 400 and 500 users confirms the CPU ceiling was reached. Crucially, the circuit breaker prevented cascading failures: HTTP error rate was 0% across all runs - the service degraded gracefully rather than crashing.

Conclusion: Single-instance capacity is ~400–500 concurrent streams on 0.25 vCPU. The CDN is the first bottleneck, not the streaming logic.

Limitations of analysis: The 500ms threshold is a simulation approximation. Real rebuffering depends on client buffer size and pre-fetch strategy, not purely p99 latency.

Experiment 2 - Adaptive Bitrate Switching Under CPU Saturation

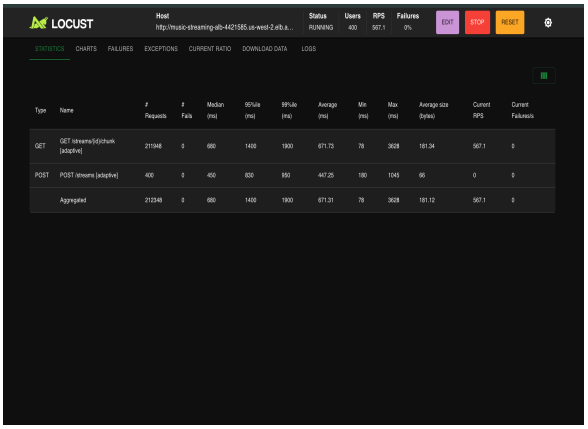
Purpose & Tradeoff: Test whether adaptive quality switching (320kbps -> 128kbps when delivery latency > 300ms) reduces rebuffering under CPU pressure. The tradeoff is **audio quality vs. streaming continuity**: dropping to 128kbps is less noticeable to a listener than a buffer stall. All runs use songs outside the CDN cache (exp2_track_*) to force local generation throughout, keeping CPU saturated - the exact worst-case scenario adaptive mode is designed for.

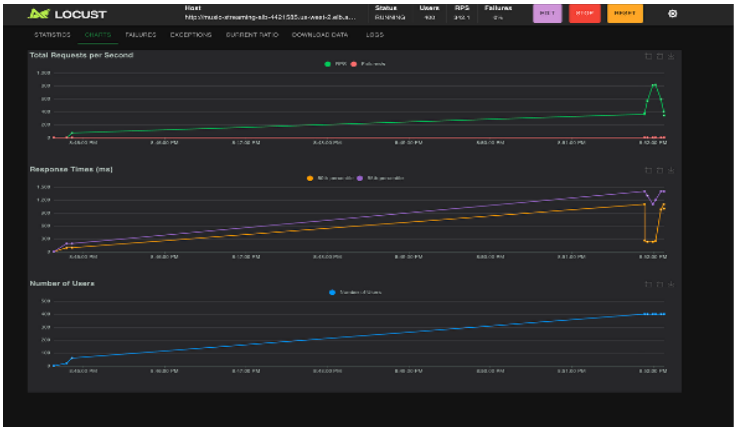
Limitations: Isolated runs only (all-adaptive vs. all-fixed). Mixed loads were also tested but masked the adaptive benefit, since CPU savings from adaptive streams were immediately absorbed by co-located fixed streams. Results do not reflect real-world mixed traffic.

Results

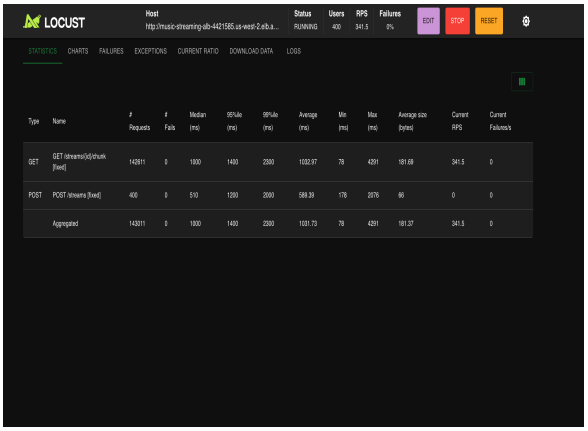
Both runs: 400 users, 7-minute duration, CDN circuit open throughout.

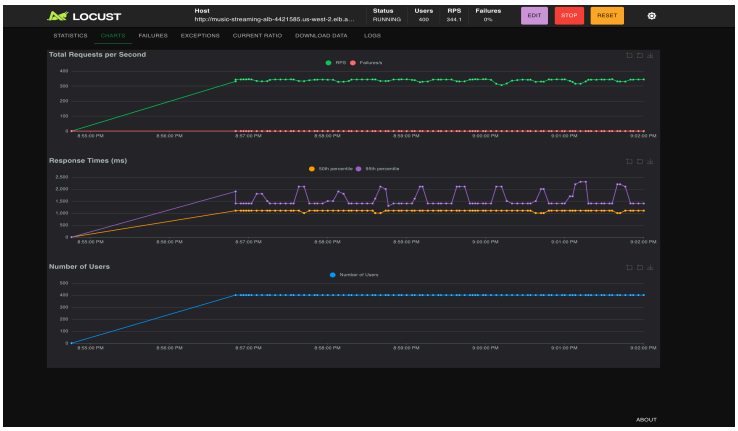
Metric	Adaptive	Fixed	Difference
Median latency	680 ms	1,000 ms	**32% faster**
p95 latency	1,400 ms	1,400 ms	same
p99 latency	1,900 ms	2,300 ms	**17% faster**
Throughput (RPS)	567.1	341.5	**+66% more**
Quality switches	~6,382 (~16/stream)	0	-



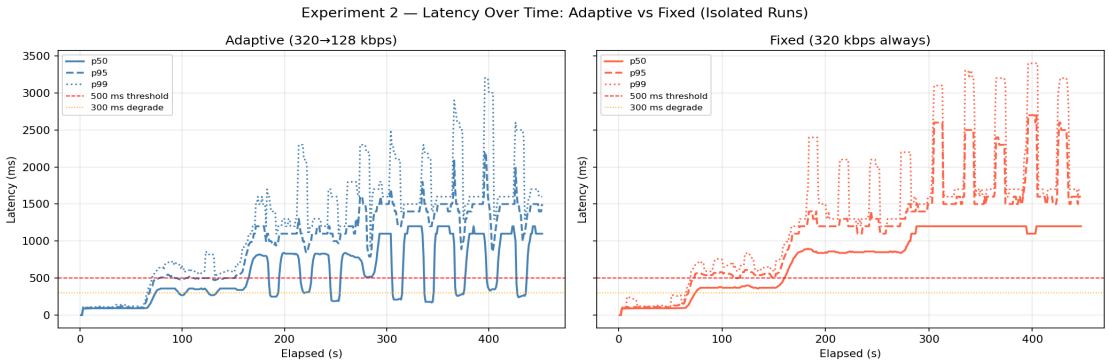


Adaptive stats and charts





Fixed stats and charts



Latency comparison (adaptive left, fixed right)

Analysis

Adaptive mode delivered 32% lower median latency and 66% higher throughput on identical hardware. The mechanism is direct: each quality switch cuts the CPU cost of that stream's next chunk from 500k to 50k iterations, freeing CPU for other streams. The latency comparison chart shows adaptive streams oscillating around a lower p50 baseline while fixed streams plateau at a persistently higher level. Rebuffer *event counts* were similar (~670% per stream in both modes), which is expected: both modes cross the 300ms threshold under saturation. The adaptive advantage is that chunks arrive sooner *after* a stall, the next chunk is 10× cheaper to generate.

Conclusion: Adaptive bitrate meaningfully improves throughput and median latency under CPU saturation. It does not eliminate rebuffering - it reduces recovery time after each stall.

Limitations of analysis: Quality switch threshold (300ms) and quality levels (320/128kbps) are fixed parameters. A tunable threshold or multi-level ladder (320->192->128) might yield better results.

Experiment 3 - Horizontal Scaling

Purpose: Test whether horizontal scaling produces near-linear capacity gains and whether it matters which tiers are scaled. Part A adds streaming instances (1->2->4) while keeping CDN fixed, predicting sub-linear gains as the single CDN becomes a bottleneck. Part B co-scales both tiers (1+1->2+2->4+4), predicting near-linear gains by distributing CDN load.

Limitations: In-memory stream state requires ALB sticky sessions - true round-robin is not achievable without shared state (e.g., Redis). CDN instances pre-populate independently, so memory cost scales linearly.

Part A - Scale Streaming Only

All runs: 1,000 concurrent users, fixed 320kbps, 7-minute duration. ALB sticky sessions (lb_cookie) required - without them, 33% of chunk requests failed with 404 because stream state is in-memory per instance.

Streaming Instances	p50 (ms)	p95 (ms)	p99 (ms)	RPS	Scaling factor
1	2,700	4,600	7,400	339.2	1×
2	1,500	2,600	3,200	552.6	1.6×
4	580	1,100	1,400	1,186.7	3.5×

Part A – Analysis

Analysis: The CDN circuit breaker opened in all three runs - the single CDN is always the bottleneck. Despite this, p50 dropped 78% (2,700→580ms) and RPS nearly tripled (3.5×). The gains come from CPU scheduling: fewer streams per instance means each goroutine gets more CPU time for local generation. Gains are sub-linear on latency but near-linear on RPS.

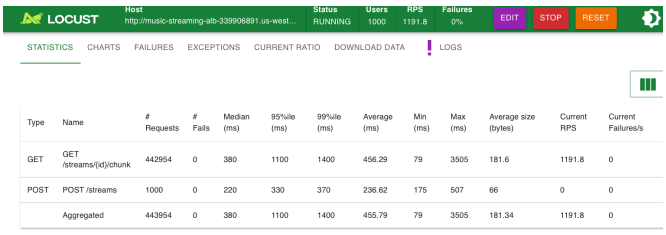
Conclusion: Scaling streaming alone produces real but sub-linear gains. CDN must scale alongside streaming to push median latency below the rebuffer threshold.

Limitations: Single CDN across all runs means CPU scheduling improvement and CDN bottleneck contribution are intertwined. CDN scaling alone does not fully resolve the bottleneck at 0.25 vCPU per instance.

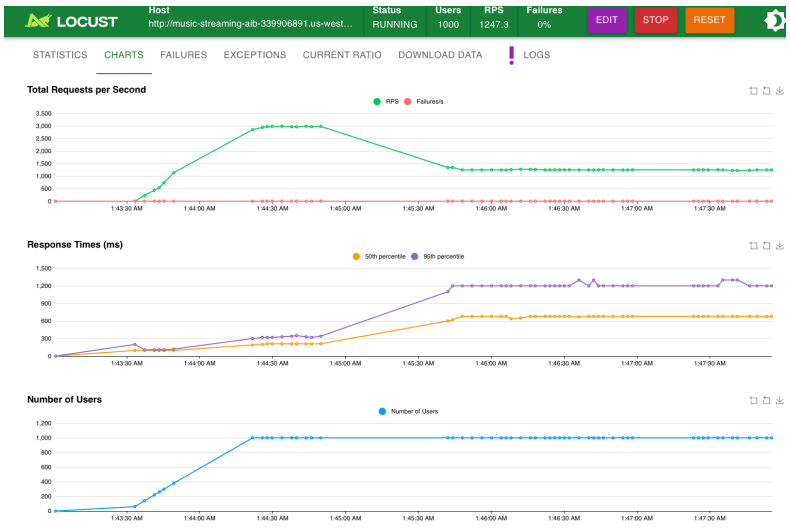
Part B - Scale Both Tiers Together

Co-scaling CDN with streaming was predicted to distribute chunk request load across N CDN instances, keeping each below the timeout threshold and the circuit breaker closed - serving chunks from cache rather than generating them locally.

Run 3 – 2+2 and 4+4 (Streaming + CDN)



Statistics and Charts



Result: At 2+2, the circuit oscillated - tripping, resetting after 30s, and briefly serving chunks from cache before getting overwhelmed again, visible as sharp latency dips in the chart. At 4+4, the circuit opened during ramp-up and remained open throughout (confirmed by server metrics mid-run). The p50 improvement (380ms) comes from the same CPU scheduling mechanism as Part A: with ~250 streams per instance, each goroutine gets more CPU time for local generation - enough to push median latency below the 500ms rebuffer threshold for the first time across all experiments.

Part B - Analysis

Config	p50 (ms)	p95 (ms)	RPS	Circuit Breaker
1+1	2,700	4,600	339	Open (always)
2+2	1,100	2,400	560	Open (oscillating)
4+4	380	1,100	1,192	Open

Analysis: Co-scaling both tiers improved outcomes over Part A, though not through the predicted mechanism. The CDN circuit breaker opened in all Part B runs - at 2+2 it oscillated, with brief cache-serving windows during resets visible as latency dips; at 4+4 it opened steadily with no oscillation. The primary driver of improvement is CPU scheduling: fewer streams per instance means faster local generation. CDN co-scaling reduced per-instance request pressure, contributing additional cache hits during reset windows and lowering aggregate p50.

To investigate further, we tested a 2:1 CDN ratio (4+8) and a higher circuit breaker threshold (5→10 failures). Both yielded identical results (p50≈510ms, circuit open), confirming the bottleneck is CPU capacity per CDN instance - not instance count or threshold sensitivity. Larger CDN tasks (e.g., 1 full vCPU) would be the correct fix.

Conclusion: At 4+4, median latency dropped to 380ms - below the 500ms rebuffer threshold - with 3.5× the throughput of a single instance. Both tiers must scale together to achieve this; streaming-only scaling (Part A) leaves the CDN as a shared bottleneck that caps gains.

Limitations: A stateless redesign (Redis-backed stream state) would remove the sticky sessions constraint and enable true round-robin load balancing.

Project Conclusion

This project simulated the core delivery challenge of music streaming at scale - serving fixed-size audio chunks to thousands of concurrent streams without rebuffering - using two Go microservices on AWS ECS Fargate. Three experiments measured the three main levers of streaming performance: single-instance capacity, quality adaptation, and horizontal scale-out.

Experiment	Key Question	Finding
1 - Single Instance	Breaking point on 0.25 vCPU?	~400–500 users; CDN trips first, CPU saturates second; 0% HTTP errors throughout
2 - Adaptive Bitrate	Does quality switching help?	+32% lower latency, +66% throughput on same hardware; recovery faster, stall frequency unchanged
3 - Horizontal Scaling	Near-linear gains?	Part A: 3.5× RPS, CDN always bottleneck. Part B (4+4): p50 drops below 500ms threshold - both tiers must scale together

The three experiments together reveal a consistent theme: the CDN circuit breaker is the architectural pivot point. It is the first bottleneck (Exp 1), the mechanism that makes CPU pressure the binding constraint (Exp 2), and the component whose saturation limits horizontal scaling gains (Exp 3 Part A). Co-scaling both tiers in Part B reduced per-instance load enough to push median latency below the 500ms rebuffer threshold - even without keeping the circuit fully closed. The key insight is that serving more concurrent streams within latency budget requires managing load at both the streaming and CDN tiers, not just one.